EDGELINK TECHNOLOGY

Technical Assessment

AI Developer & Automation Engineer

SEO Automation Dashboard — Full-Stack Build Assessment

⚠ READ BEFORE YOU START

Read this document end-to-end before writing a single line of code. Every architecture decision, schema, prompt, and integration is specified here. Deviating from the specified stack or folder structure without written approval will result in disqualification.

API KEYS POLICY

You will NOT be provided agency API keys. Your job is to build the full system with mock/stub integrations that can be swapped for real keys in production. The pipeline must work end-to-end with mock data where specified. Real credentials required only for Claude API, Supabase/Neon (Postgres) Atlas, and Redis — all available on free tiers.

Document Quick Reference

Role	Stack	Submission Deadline
AI Developer / Automation Engineer	Next.js 14 + Node.js + BullMQ + Claude API	3 to 4 days from date of receipt

SECTION 1
Overview & Objective

You are required to design and build a SEO Automation Dashboard. This system automates website SEO analysis using headless crawling, AI analysis engine, and a dual-portal interface (internal admin + client-facing dashboard). (If needed use mock data)

The system must handle automated audit pipelines, AI-powered recommendations, PDF report generation, and real-time notifications.

1.1 Scope Summary

Area	Details
Backend	BullMQ job queue workers (Node.js)
AI Engine	Anthropic Claude API (claude-sonnet-4-6) for audit analysis (Use mock data)
Frontend	Next.js 14 Admin Portal + Client Dashboard
Data Layer	Supabase / Neon (Postgres) + Upstash Redis
Integrations	Google PageSpeed Insights, GSC, DataForSEO, Twilio WhatsApp (use mock data)
Output	Automated PDF audit reports delivered via WhatsApp + Email

SECTION 2
API Keys Policy & Mock Strategy

2.1 What to Use Real vs Mock

The table below is the single source of truth for what requires a real account versus what must be mocked. Do not deviate from this.

Integration	Status	Reason
Anthropic Claude API	☑ REAL/ MOCK	Core to the role. Free $5 starter credit on new API account at console.anthropic.com. Covers 10–15 full audit test runs on Sonnet 4.6. / use mock data
Supabase/Neon (Postgres) Atlas	☑ REAL	Free M0 tier available. Candidate sets up own account.
Upstash Redis	☑ REAL	Free tier: 500K commands/month, 256MB. No local server needed — fully serverless.
Google PageSpeed Insights	MOCK	Rate-limited. Realistic mock data provided in this document.
Google Search Console	MOCK	OAuth + domain verification is complex. Mock completely.
DataForSEO	MOCK	Costs real money per API call. Mock data provided.
Twilio WhatsApp	MOCK	Requires approved business template. Log to console instead.
Cloudflare R2	MOCK	Replace with local /tmp/reports/ folder for testing.
cron-job.org	☑ REAL	Free external cron scheduler. Triggers your Vercel API route every minute. Sign up at cron-job.org.

2.2 Mock Architecture — Required Implementation

You must implement a clean toggle switch. All mock files live in a /src/mocks/ folder.

Required Folder Structure

```
// code
src/
├── mocks/
│   ├── index.js           ← USE_MOCKS toggle switch
│   ├── pagespeed.mock.js  ← PageSpeed Insights mock data
│   ├── gsc.mock.js        ← Google Search Console mock data
│   ├── dataforseo.mock.js ← DataForSEO rankings mock data
```

```
|   └── twilio.mock.js      ← WhatsApp console.log mock
```

Toggle Switch (mocks/index.js)

A single environment flag must control all mock behaviour:

```
// code
// src/mocks/index.js
const USE_MOCKS = process.env.USE_MOCKS === 'true';
module.exports = { USE_MOCKS };
```

Integration Pattern (apply to every crawler file)

```
// code
// src/crawlers/pagespeed.js
const { USE_MOCKS } = require('../mocks');
const mockData    = require('../mocks/pagespeed.mock');

async function getPageSpeed(url) {
  if (USE_MOCKS) return mockData;
  // Real PageSpeed Insights API call goes here
}

module.exports = { getPageSpeed };
```

Environment Variable

Add this to your .env file. Setting it to true runs the full pipeline on mock data:

```
// code
USE_MOCKS=true    # set false when connecting real APIs
```

2.3 Mock Data Files — Use These Exactly

Copy the following mock data files as-is. This ensures evaluation is consistent across candidates.

pagespeed.mock.js

```
// code
module.exports = {
  url: 'https://example-dental-clinic.com',
  strategy: 'mobile',
  categories: {
    performance:   { score: 0.61 },
    seo:           { score: 0.89 },
    accessibility: { score: 0.74 }
  },
  audits: {
    'largest-contentful-paint':  { score: 0.4, displayValue: '4.2 s' },
    'cumulative-layout-shift':   { score: 0.9, displayValue: '0.08' },
    'interaction-to-next-paint': { score: 0.6, displayValue: '280 ms' },
    'render-blocking-resources': { score: 0.5, description: '2 blocking
resources' },
    'unused-javascript':         { score: 0.4, displayValue: '420 KiB' }
  }
};
```

gsc.mock.js

```
// code
module.exports = {
  rows: [
    { query: 'dental clinic in pune',     clicks: 42, impressions: 980,  ctr:
0.043, position: 6.2 },
    { query: 'best dentist pune',         clicks: 28, impressions: 740,  ctr:
0.038, position: 8.1 },
    { query: 'tooth extraction pune',     clicks: 19, impressions: 430,  ctr:
0.044, position: 5.7 },
    { query: 'dental implants cost pune', clicks: 11, impressions: 310,  ctr:
0.035, position: 11.3 },
    { query: 'dentist near me',           clicks: 67, impressions: 2100, ctr:
0.032, position: 9.4 }
  ],
  totalClicks: 167,
  totalImpressions: 4560,
```

```
  indexCoverageErrors: 3
};
```

dataforseo.mock.js

```
// code
module.exports = {
  tasks: [{
    result: [{
      keyword: 'dental clinic pune',
      items: [
        { type: 'organic', rank_group: 1, rank_absolute: 4,  domain: 'example-dental-clinic.com' },
        { type: 'organic', rank_group: 2, rank_absolute: 8,  domain: 'competitorA.com' },
        { type: 'organic', rank_group: 3, rank_absolute: 12, domain: 'competitorB.com' }
      ]
    }]
  }],
  client_rank_position: 4,
  keyword_search_volume: 1900
};
```

twilio.mock.js

```
// code
async function sendWhatsApp(clientId, pdfPath, score) {
  console.log('[MOCK WHATSAPP] → Client:', clientId);
  console.log('[MOCK WHATSAPP] → Score: ', score + '/100');
  console.log('[MOCK WHATSAPP] → Report:', pdfPath);
  return { status: 'mock_sent', sid: 'MOCK_SID_' + Date.now() };
}

module.exports = { sendWhatsApp };
```

SECTION 3
System Architecture

3.1 Technology Stack —If Any other Change According

Layer	Component	Technology
Presentation	Admin Portal	Next.js 14 App Router + Tailwind CSS + shadcn/ui
Presentation	Client Dashboard	Next.js 14 App Router + Tailwind CSS + shadcn/ui
Application	Audit Job Runner	BullMQ + Redis (VPS)
Application	Headless Crawler	Fetch + Cheerio (serverless-compatible crawler)
Application	AI Analysis Engine	Anthropic Claude API (claude-sonnet-4-6)
Application	Report Generator	Puppeteer PDF (primary) or WeasyPrint (fallback)
Data	Audit Results	Supabase (Postgres) or Neon (Postgres)
Data	Job Queue	Upstash Redis (BullMQ)
External APIs	Performance Data	Google PageSpeed Insights API (MOCKED)
External APIs	Search Data	Google Search Console API (MOCKED)
External APIs	SERP + Keyword Data	DataForSEO (MOCKED)
Delivery	Report Delivery	Twilio WhatsApp + Email (MOCKED)
Hosting	Frontend + API + Cron	Vercel (free hobby tier)
Hosting	Backend Workers	Vercel (serverless functions + cron)

3.2 Audit Job Pipeline — Required Flow

Every audit must follow this exact pipeline in sequence.

```
// code
TRIGGER (manual admin click / monthly cron)
     |
     v
BullMQ QUEUE: seo-audit-jobs  [Upstash Redis (serverless)]
     |
     v
WORKER picks up job  [Node.js on VPS]
     |
     |--- Step 1:  Fetch + Cheerio crawls URL
     |--- Step 2:  PageSpeed Insights API  (MOCKED)
```

```
      |--- Step 3:  Google Search Console   (MOCKED)
      |--- Step 4:  Schema.org validator
      |--- Step 5:  On-page analysis
      |--- Step 6:  robots.txt + sitemap.xml check
      |--- Step 7:  DataForSEO rankings      (MOCKED)
      |
      v
Claude API: analyse all signals -> structured JSON recommendations  (REAL)
      |
      v
Supabase/Neon (Postgres): store audit result  (REAL)
      |
      v
PDF Report Generator (Puppeteer)
      |
      v
Twilio WhatsApp -> console.log mock
```

SECTION 4

Project Setup — All Free, Zero Cost

4.1 Free Platform Accounts — Create These First

All platforms below are free. No credit card required for any of them. Verified June 2026.

Platform	Free Tier Limits (Verified June 2026)	Notes
Vercel vercel.com	Free Hobby: 100GB bandwidth, 1M function invocations/month, 10s function timeout, 4hr Active CPU/month.	Hobby plan is personal/non-commercial. 10s function timeout is the key limit — keep all API routes fast. Use cron-job.org for sub-hour cron triggers.
cron-job.org cron-job.org	Free. Unlimited jobs. Per-minute granularity.	Use this instead of Vercel cron to trigger your audit worker API route every minute. No signup friction.
Upstash Redis upstash.com	Free: 500K commands/month, 256MB storage. 1 database.	Serverless Redis for BullMQ. No server needed. Stable pricing — no changes recorded in 2026.
Supabase supabase.com	Free: 500MB database, 1GB file storage, 5GB bandwidth, 50K monthly active users, 2 active projects.	Projects PAUSE after 7 days of inactivity — add a scheduled keep-alive ping or use Neon instead.
Neon neon.tech (alternative)	Free: 500MB storage, auto-scaling, branching. No inactivity pause.	Better choice if you want to avoid Supabase's 7-day pause. Acquired by Databricks in 2025 — pricing improved.

SECTION 5

Database Schema — Prisma + Supabase / Neon

Implement the following Prisma schema exactly. Run npx prisma db push to sync it to your Supabase or Neon database.

SECTION 6

Audit API Route — Required Implementation

7.2 Master SEO Audit Prompt

System Prompt

```
// code
You are a senior SEO analyst with 15+ years experience specialising in
Indian local business websites, healthcare, dental, and real estate sectors.
You produce honest, falsifiable, prioritised SEO audit reports.
Your output must be valid JSON only — no markdown, no preamble.
Every recommendation must include: finding, fix, estimated_impact
(High/Medium/Low),
effort (Hours/Days/Weeks), falsifiability_check (how we know if this worked).
```

User Prompt Template — Replace All {VARIABLES} at Runtime

```
// code
Analyse the following SEO signals for: {URL}
Business type: {BUSINESS_TYPE}
Business location: {CITY}, {STATE}, India
```

```
=== ON-PAGE DATA ===
{ON_PAGE_JSON}

=== TECHNICAL DATA ===
{TECHNICAL_JSON}

=== CORE WEB VITALS ===
{CWV_JSON}

=== SCHEMA.ORG DATA ===
{SCHEMA_JSON}

=== GOOGLE SEARCH CONSOLE ===
{GSC_JSON}

=== KEYWORD RANKINGS ===
{RANKINGS_JSON}

Respond ONLY with this JSON structure — no other text:
{
  "audit_version":    "v1.0",
  "overall_score":    <integer 0-100>,
  "score_breakdown": { "technical": <0-100>, "on_page": <0-100>,
                       "content": <0-100>, "core_web_vitals": <0-100>, "schema":
<0-100> },
  "critical_issues":  [<max 5 items>],
  "important_issues": [<max 8 items>],
  "quick_wins":       [<max 5 items — effort <= 2 hours>],
  "recommendations":  [{
    "priority":             "CRITICAL|HIGH|MEDIUM|LOW",
    "category":             "Technical|OnPage|Content|Schema|CWV",
    "finding":              "<what is wrong>",
    "fix":                  "<exact action to take>",
    "estimated_impact":     "High|Medium|Low",
    "effort":               "Hours|Days|Weeks",
    "falsifiability_check": "<how to verify this worked>"
  }],
  "executive_summary": "<3 sentences max, plain English>"
}
```

SECTION 7
Deliverables & Submission Checklist

7.1 How to Submit

Send your completed submission to:

hr@edgelinktechnology.com

Use this subject line:

```
Technical Assessment — AI Developer — [Your Full Name]
```

Your email must include all 3 items below:

Item	Details
GitHub Repo	Public or private repository link. If private, add hr@edgelinktechnology.com as a collaborator. Must include all source code, README.md, and .env.example.
Video Recording	3 to 5 minute screen recording of the full audit pipeline running end-to-end. Show: job queued in BullMQ → progress updating in admin portal → Claude returning JSON → audit saved in Supabase/Neon (Postgres) → PDF generated. Upload to Loom, Google Drive, or YouTube (unlisted) and paste the link in your email.

Deadline: 3 to 4 days from the date you receive this document.